Civic Operations

Explainable City Service Triage and Field Crew Dispatch

Group Project Report

Practice Module, Certificate in Intelligent Reasoning Systems

NUS Institute of Systems Science, Master of Technology in Artificial Intelligence Systems

Group 23 · Submitted 25 October 2026

Member	Student ID	Components owned
Khalid Alhazmi	A0361128J	Urgency, Prioritisation
Mohammad Alharthi	A0354097W	Agency and skill routing, External factors, Crew deployment and routing
Nabeel Taresh	A0352296X	Duplicate detection, Fake request detection, Admin questions
Indish Sankalp Sandeep	A0351383E	Intake bot, Emergency check, Credibility and escalation, Predictive alerting, application and integration

Executive summary

New York City's 311 service receives millions of non emergency requests a year: potholes, blocked drains, broken street lights, noise. Each one has to be read, sent to the right agency, ranked against everything else that is open, and given to a crew. Today that triage is largely manual and the reasons behind it are invisible to the resident who reported the problem.

Civic Operations is a working system that does this triage with intelligent reasoning and explains every step. A resident reports a problem in plain words or through a short form. Twelve AI components then read it, stop anything that belongs with 911, route it to an agency, link duplicates, check nearby schools and hospitals, weigh the reporter's track record and the risk that a report is unfounded, assign an urgency, rank it against every open request, and plan crew routes along the real road network. City staff approve the plan, crews record what they found, and that outcome flows back into the reporter's record and to the resident.

The system runs as four web portals over one API in two Docker containers, on 150 real NYC 311 requests. Every component sits behind an adapter with an agreed safe default, so a failure never stops the service, and the tech team can switch any component off and see the effect. Measured on 34 hand written scenarios and their second reports, switching off Duplicate detection changes the queue order to a Kendall tau b of 0.637, External factors to 0.969, and Prioritisation to 0.358. The crew planner's genetic algorithm serves 9 to 18 percent more requests than dispatching strictly in queue order while cutting travel by 38 to 48 percent. People keep the decisions that carry risk: escalations, confirming an impact on a school or hospital, and sending routes to crews.

Source code and run instructions: the GitHub repository, folder `01_Application`, with `SETUP.md` for a five command setup. Repository: https://github.com/DevelopingGod/IRS-Project-2026-KIMS

Team and contributions

The four of us worked from Singapore and Saudi Arabia. From the project start on 5 September we held 14 MS Teams meetings of over an hour each, kept a shared tracker, and agreed every component's inputs and outputs in a written specification before building. The log book attached to this submission records the work by date.

Table 1. Who did what

Member	Work items
Khalid Alhazmi	Urgency (forward chaining rules over hazard and service impact knowledge bases) and Prioritisation (weighted multi criteria ranking with ageing, SLA and duplicate rules). An urgency learning experiment across four model families. 33 commits.
Mohammad Alharthi	Agency and skill routing (v1.1.0), External factors (OpenStreetMap proximity, v1.0.0) and Crew deployment and route optimisation (genetic algorithm with A* routing, v1.0.0), each with its own research report and test suite. The GPU run of the language model experiments. Independent label check.
Nabeel Taresh	Duplicate detection (hybrid evidence matcher tuned on 295 hand checked pairs), Fake request detection (calibrated rules and logistic regression) and Admin questions (natural language questions over the live queue, read only). The first data flow diagram. 13 commits.
Indish Sankalp Sandeep	Intake bot, Emergency check, Credibility and escalation, Predictive alerting. The data pipeline, component specification and team guides. The application: backend, four portals, accounts, integration of all twelve components, Docker, testing and measurement. 72 commits.

Contents

Executive summary 2

Team and contributions 2

1 Business case 6

1.1 The problem 6

1.2 Market and literature review 6

1.3 Value and who benefits 6

1.4 Objectives and scope 6

2 System design 7

2.1 Architecture 7

2.2 How a request flows 9

2.3 The twelve components 9

2.4 Design decisions 10

2.5 Data 10

2.6 Privacy, security and safety 10

3 Development and implementation 11

3.1 Technology 11

3.2 The four portals 11

3.3 Planning and delivery 12

3.4 What integration taught us 13

4 Experiments 13

4.1 Results at a glance 13

4.2 Intake bot 14

4.3 Emergency check 14

4.4 Credibility and escalation 15

4.5 Predictive alerting 15

4.6 Duplicate detection 15

4.7 Fake request detection 16

4.8 Admin questions 16

4.9 Agency and skill routing 17

4.10 External factors 17

4.11 Urgency and Prioritisation 18

4.12 Crew deployment and route optimisation 18

4.13 The experimentation journey 19

4.14 Does each component matter? Switching them off 20
4.15 Testing 20
5 Findings and discussion 20
5.1 What worked 20
5.2 Limitations 21
5.3 Ethics 21
5.4 Future work 21
6 Conclusion 21
References 21
Glossary 22
Appendix A Functions mapped to MR, RS and CGS 22
Appendix B Installation and user guide 23
Installation 23
Using it 23
Appendix C Project proposal 23

Figures

Figure 1. Architecture: four portals, one API, twelve components in the order a request meets them 8
Figure 2. One request from report to closure, in nine steps 9
Figure 3. Request types in the real 311 sample 10
Figure 4. Resident portal: the home page with the 3D city (left), and reporting in plain words to the Intake bot (right) 11
Figure 5. Tracking a request with the status bot (left), and the Emergency check stopping a report (right) 11
Figure 6. City staff: the ranked queue, each card saying why the AI ranked it (left), and a request's reasons step by step (right) 12
Figure 7. Crew deployment's proposed route on the road network for staff to approve (left), and the tech team's component switches (right) 12
Figure 8. Planned and actual timeline 12
Figure 9. Commits per member per week on GitHub, all branches. Mohammad delivered his packages as files, so his work shows as deliveries rather than commits. 13
Figure 10. Intake bot: request type accuracy on two test slices (left), slot span F1 against the pattern baseline (right) 14
Figure 11. Emergency check: recall and false alarms across thresholds (left), recall by emergency category (right) .. 14
Figure 12. Credibility: score against each reporter's hidden honesty (left), and fairness across report types (right) .. 15
Figure 13. Predictive alerting: error of each forecasting approach (left), storm day spikes caught (right) 15
Figure 14. Duplicate detection: F1 of each method on the same 295 hand checked pairs 16
Figure 15. Fake request detection: AUC of each method on 5,390 held out requests 16
Figure 16. Admin questions: three strategies on 74 gold questions 17
Figure 17. External factors: threshold candidates on validation data (left), requests flagged across the city (right) .. 18

Figure 18. Urgency: macro F1 of each learned model (left), and the maximum points each factor adds in Prioritisation (right) 18
Figure 19. Crew deployment: the genetic algorithm against two greedy baselines, capacity factor 1.0 19
Figure 20. Crew deployment: methods across crew capacity (left), and one planned day of routes on real roads (right) 19
Figure 21. Queue order with each component on its safe default, as Kendall tau b against everything on 20

Tables

Table 1. Who did what 2
Table 2. What exists and what does not 6
Table 3. Value to each user 6
Table 4. Components, techniques and owners 9
Table 5. Decisions that shaped the system 10
Table 6. Technology stack 11
Table 7. Integration problems found and fixed 13
Table 8. Headline results per component 13
Table 9. Agency and skill routing: approaches compared 17
Table 10. Experiments that changed a decision 19
Table 11. System functions against the course knowledge and techniques 22

1 Business case

1.1 The problem

311 lines exist so routine problems never compete with life safety calls, a split New York and most large cities have kept since the late 1990s. The volume is the difficulty. Our own sample of NYC Open Data holds 49,558 requests from five boroughs over a few weeks. Each needs four decisions: is this really an emergency, which agency owns it, how urgent is it compared with everything else, and which crew should go, in what order. Getting any of these wrong costs money or safety: a wasted crew trip to a duplicate or an unfounded report, a broken pavement outside a school left behind newer and easier jobs, or a crew crossing the borough twice in a day.

Residents feel the second cost. Once a request is filed it usually disappears into a queue they cannot see, so the same problem gets reported again and again, which adds load to the very queue that is slow.

1.2 Market and literature review

Table 2. What exists and what does not

What we reviewed	What it does	What is missing
311 CRM platforms: SeeClickFix (CivicPlus), CitySourced (Granicus), Accela	Multi channel intake, category routing, service level timers for escalation	No reasoning over a request's context, no explanation to staff or residents, no route optimisation
Academic and capstone 311 work (for example IBM 311 capstones, NYC demand forecasting studies)	Classify a complaint, or forecast demand, as a single task	None chains prediction into policy rules, screening, ranking, crew dispatch and a resident status loop
Vehicle routing research (genetic algorithms, A* search)	Efficient routes for a fixed set of jobs	Ignores the priority a city assigns, and replans without regard to work a person already approved

The gap is the chain. Each piece exists somewhere, but no system we found joins them so that each decision feeds the next and every one can be explained. That chain, with a person approving the risky steps, is what Civic Operations adds.

1.3 Value and who benefits

Table 3. Value to each user

User	Value	How it shows in the system
Residents	Know their report was heard, where it is, and why	A tracking page and a status bot that answers in plain words
City staff	A ranked queue with reasons, and routes ready to approve	Every rank says why, routes are drawn on the road map
Crews	Fewer wasted trips and a sensible order of stops	Duplicates linked, unfounded reports flagged, optimised routes
The city	Faster fixes for what matters most, measurable decisions	Every component call timed, switchable and measured

1.4 Objectives and scope

- Build a runnable system, not a set of notebooks: four portals, one API, every component live.
- Keep a person in control of every decision with a real cost: escalations, impact on sensitive sites, sending crews.
- Make every decision explainable: each component returns its reasons, and staff see them beside the result.
- Measure each component on its own and inside the system, and report the result honestly, including what did not work.

- Out of scope: live integration with a real city, real resident identities (all accounts are pseudonymous), and two optional components from the proposal, Resolution time and the Profanity filter, which the team dropped to finish the rest well.

2 System design

2.1 Architecture

Four portals share one API. The API runs the twelve components in four stages, each component behind an adapter that renames shared fields, calls the owner's code, and falls back to the safe default agreed in the component specification when the code is missing, fails, returns the wrong shape or is switched off.

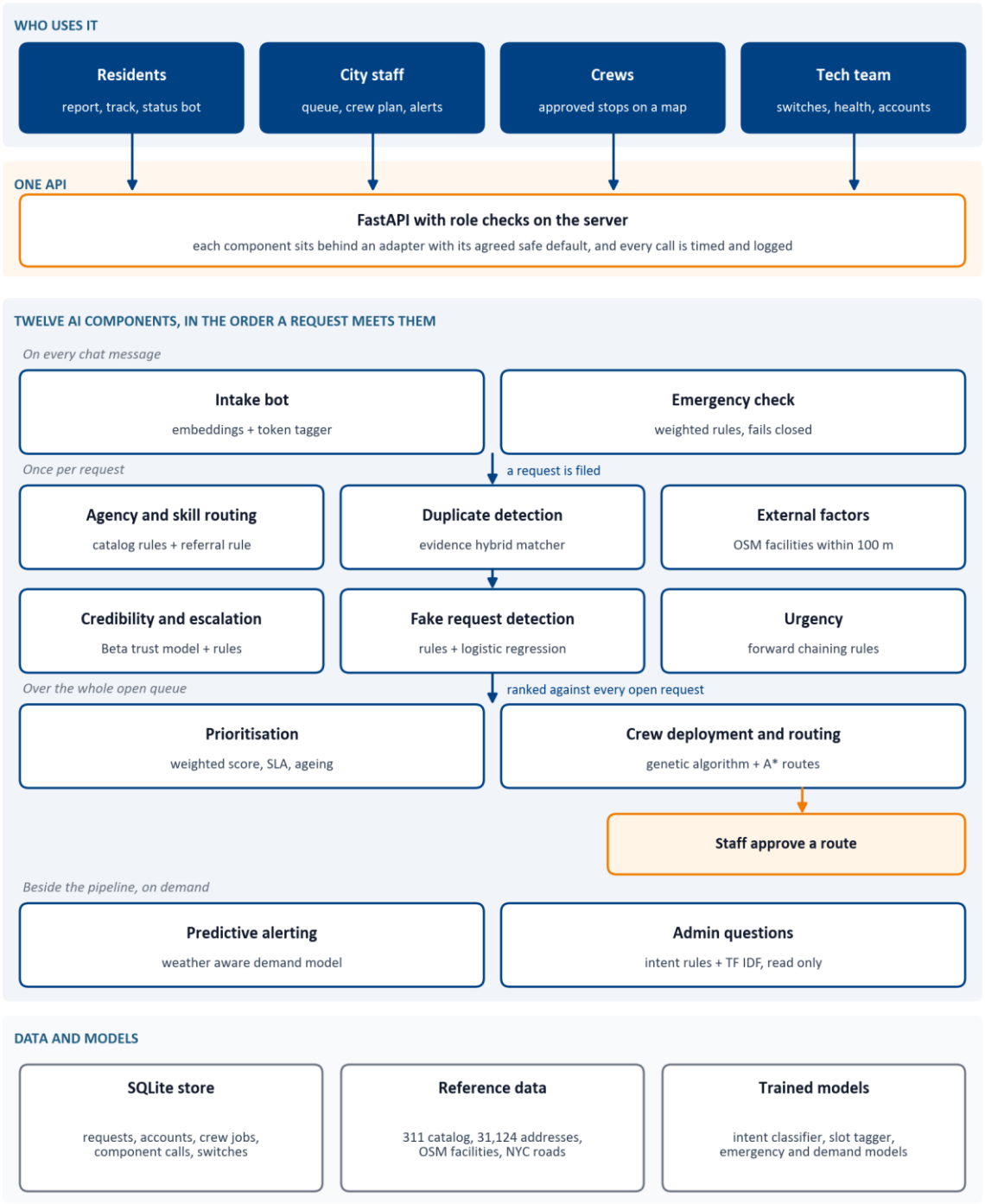


Figure 1. Architecture: four portals, one API, twelve components in the order a request meets them

Read top to bottom: who uses it, the one API, the components in four stages, then the data. Orange marks the step a person approves.

2.2 How a request flows

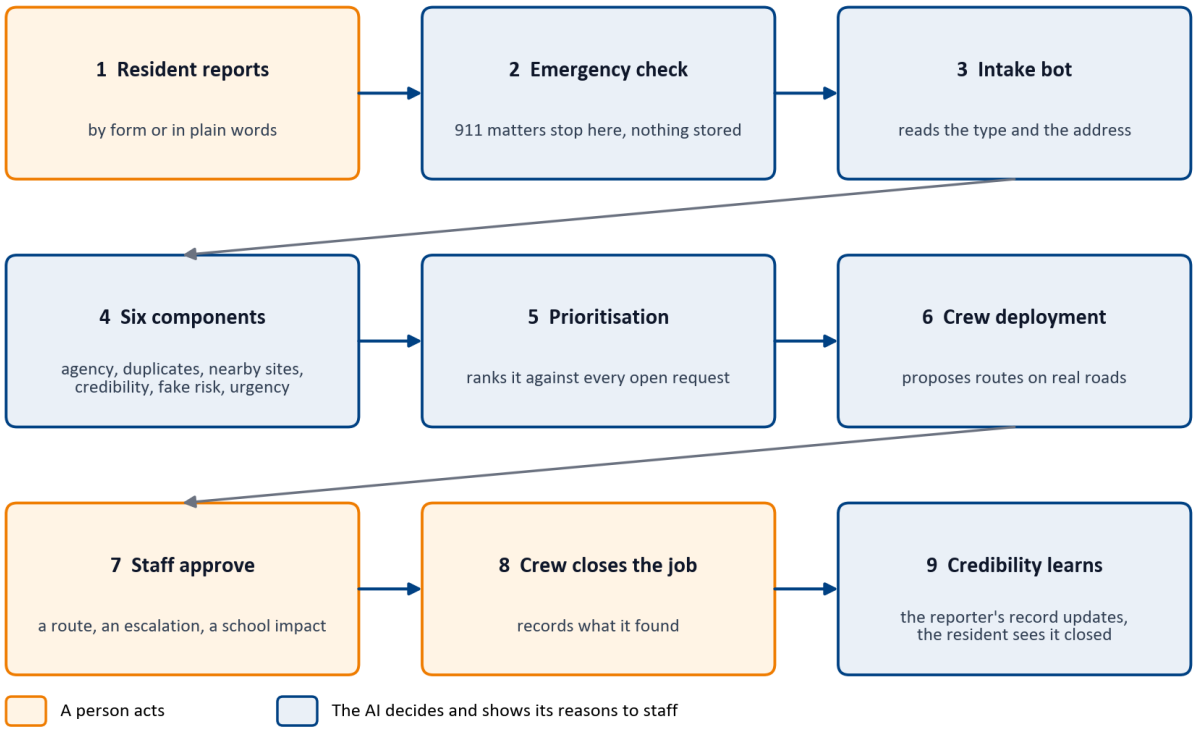


Figure 2. One request from report to closure, in nine steps

Read left to right, row by row. People act at the orange steps and the AI at the blue ones, always showing staff its reasons.

2.3 The twelve components

Table 4. Components, techniques and owners

Component	What it decides	Technique	Owner
Intake bot	Request type and address from plain words	Sentence embeddings with logistic regression, fine tuned token tagger for slots	Sankalp
Emergency check	Stop anything that belongs with 911	Weighted lexicon rules with a confirmation question, fails closed	Sankalp
Agency and skill routing	Agency, crew skill, hand off advisory	Catalog rules plus a train derived referral rule	Mohammad
Duplicate detection	Same incident as an open request, or a recurrence	Hybrid evidence matcher over time, distance and address	Nabeel
External factors	A school, hospital or similar within 100 m	OpenStreetMap facilities, geodesic distance rules	Mohammad
Credibility and escalation	Reporter track record, escalation advice	Beta distribution trust model, rule engine	Sankalp
Fake request detection	Risk that a report is unfounded	Calibrated rules and logistic regression	Nabeel
Urgency	Critical, high, medium or low	Forward chaining rules over hazard knowledge bases	Khalid
Prioritisation	Place in one ranked queue	Weighted multi criteria score with ageing and guardrails	Khalid
Crew deployment and routing	Which crew, in what order, on which roads	Genetic algorithm, A* search with ALT landmarks	Mohammad
Predictive alerting	Which report types will rise today	Weather aware demand model against a seasonal baseline	Sankalp
Admin questions	Answers to staff questions about the queue	Intent rules and TF IDF classifier, read only queries	Nabeel

2.4 Design decisions

Table 5. Decisions that shaped the system

Decision	Why
One adapter and one safe default per component	Four people built components separately. The contract let each one plug in, and a failure never stops a resident or a crew.
People approve the risky steps	An escalation, a sensitive site impact and a route all cost money or attention. The AI proposes, a person decides, and the decision is recorded.
Proposal only crew planning	The planner never assigns. An approved job is locked to its crew and is never planned again, so plans stay stable.
Rules where the data is the rules	The urgency learning experiment reached near perfect scores because it imitated the catalog policy, so the explainable rules ship.
Pseudonymous accounts	Residents register with an email only and receive an ID. Only a keyed hash of the email is kept.

2.5 Data

Real data: 49,558 NYC 311 requests across the five boroughs from NYC Open Data, 31,124 distinct addresses built from them, 4,445 facilities from OpenStreetMap and the NYC drive network for routing, and daily weather. Synthetic data, always labelled as such: reporter identities and their histories, crew rosters with depots, and hand written test sets (308 intake messages, 74 Admin questions, 295 duplicate pairs labelled by hand). The app starts from a real backlog: the 30 most recent requests in each borough, run through every component.

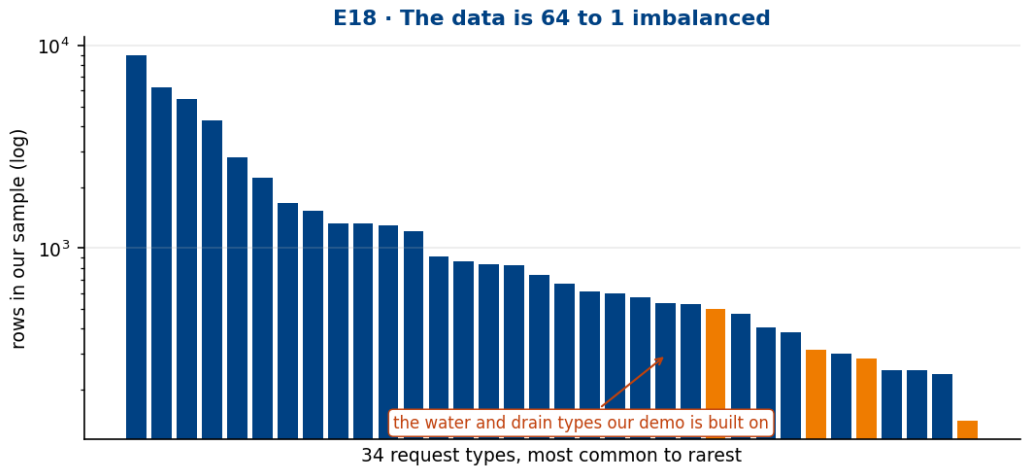


Figure 3. Request types in the real 311 sample

A few request types dominate and many are rare, so every evaluation reports macro scores that weigh rare types equally.

2.6 Privacy, security and safety

- Emergencies are never filed: the conversation stops with a full screen instruction to call 911, and no words are stored.
- Four roles, checked on the server: residents, city staff, crews and the tech team. Staff PINs are salted PBKDF2 hashes, and five wrong PINs lock an ID.
- Staff never see who reported a request, only the reporter's credibility score. A resident never sees a score.
- Security scans before each merge: bandit, pip audit, npm audit and gitleaks, all clean.

3 Development and implementation

3.1 Technology

Table 6. Technology stack

Layer	Choice
Components	Python 3.10, scikit learn, sentence transformers, PyTorch (CPU), pandas, SciPy, OSM data
API	FastAPI with role based access, SQLite store
Portals	React 19 and Vite, a 3D city with three.js, phone first layout
Deployment	Two Docker containers (API and nginx), one command start, tested from a fresh clone
Testing	168 automated tests for the app and components, browser tests at phone, tablet and desktop size

3.2 The four portals

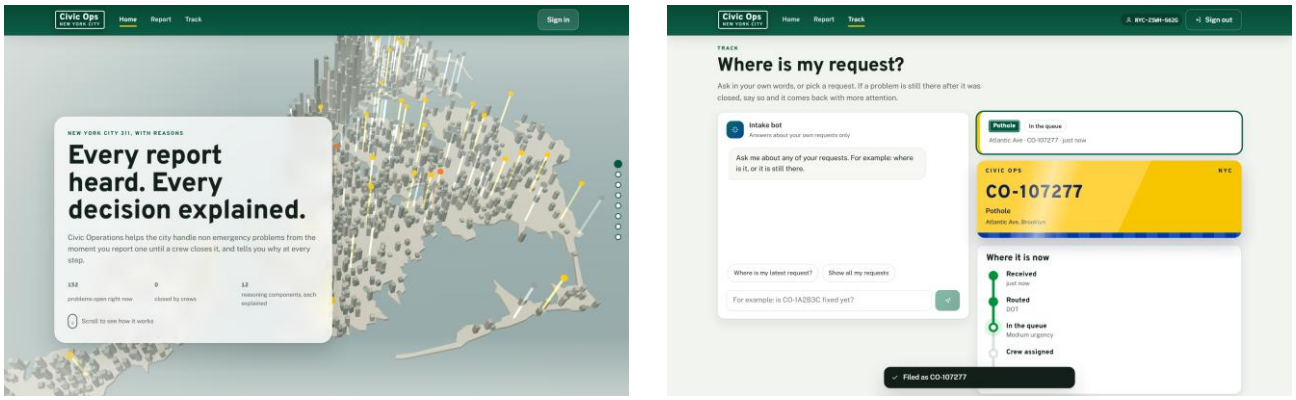


Figure 4. Resident portal: the home page with the 3D city (left), and reporting in plain words to the Intake bot (right)

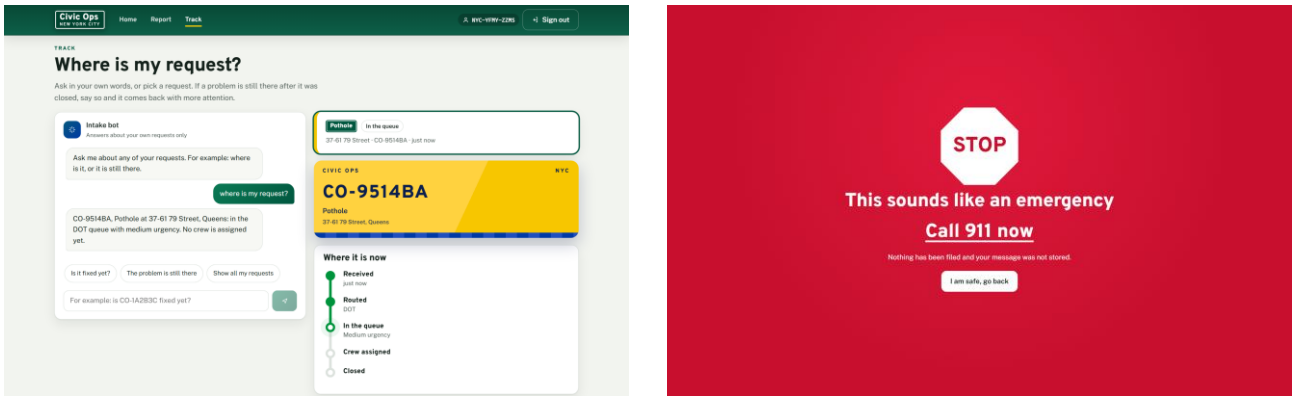


Figure 5. Tracking a request with the status bot (left), and the Emergency check stopping a report (right)

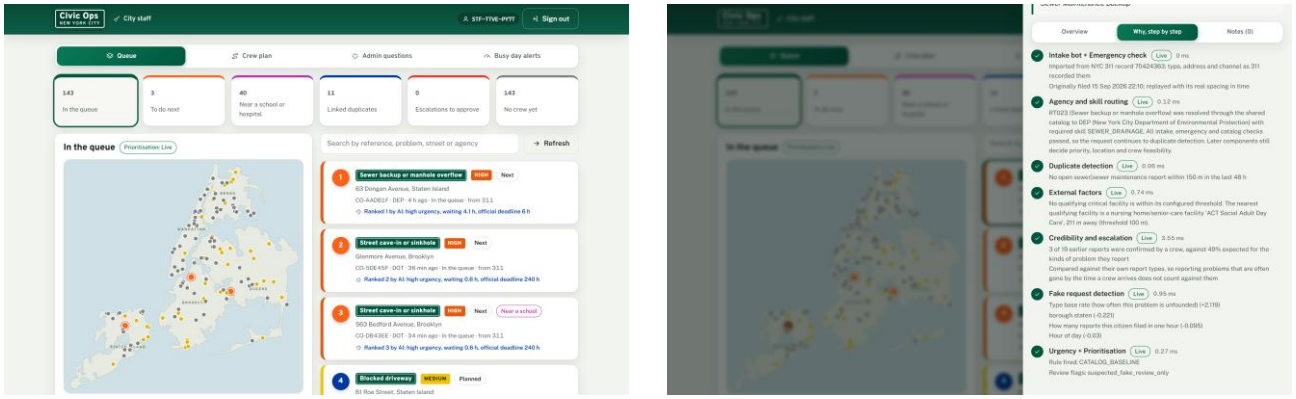


Figure 6. City staff: the ranked queue, each card saying why the AI ranked it (left), and a request's reasons step by step (right)

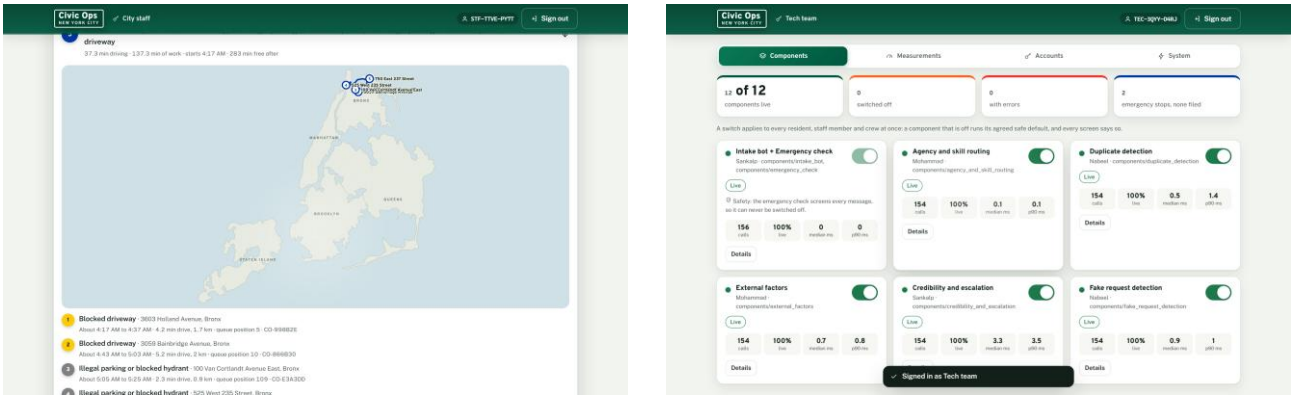


Figure 7. Crew deployment's proposed route on the road network for staff to approve (left), and the tech team's component switches (right)

3.3 Planning and delivery

We planned in three phases: define and propose (5 to 12 September), build and evaluate components separately against a written contract (to mid October), then integrate, test and measure. The chart below compares the planned windows from our tracker with the delivery dates in our git history. Components arrived one to ten days after their planned dates, and the contract made late arrivals cheap: each one plugged into an adapter that was already running on its safe default.

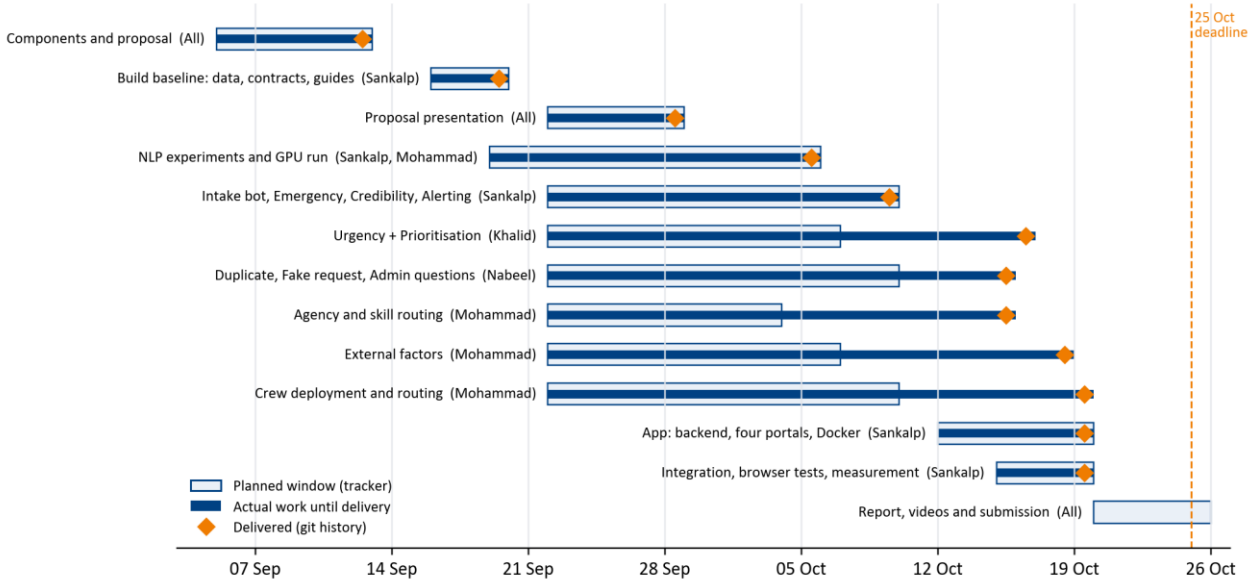


Figure 8. Planned and actual timeline

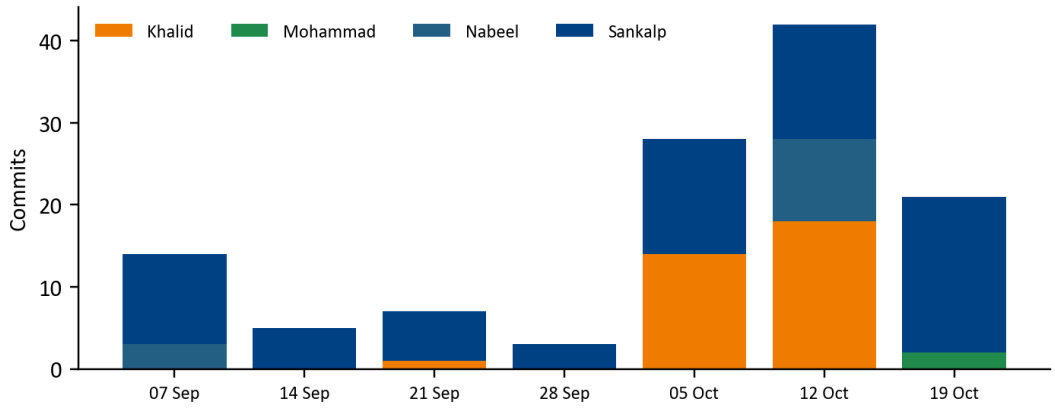


Figure 9. Commits per member per week on GitHub, all branches. Mohammad delivered his packages as files, so his work shows as deliveries rather than commits.

3.4 What integration taught us

Joining twelve components built by four people exposed problems no component test could see. Each was traced to a root cause, fixed, and given a test so it cannot return.

Table 7. Integration problems found and fixed

What we saw	Root cause	Fix
Two reports of the same pothole were not linked	The Intake bot cut 450 Atlantic Avenue to 450 Atlantic	Taught the address reader New York street types and corners. Hand labelled addresses went from 28 to 29 of 29
Every request showed two reporters	Requests reached Duplicate detection before they had an ID, so all shared one incident group	Give each request its ID first. Now 141 incidents, only the 8 real duplicates show two reporters
External factors never moved the queue	Prioritisation counts a sensitive site only once its impact is confirmed, and staff could not confirm it	A confirm button for staff, like the escalation approval
A fresh clone refused to plan crews	Windows line endings changed a file the crew planner checks by hash	Store the package byte for byte. Tested on a fresh clone
Crew rosters had duplicate IDs	Bronx and Brooklyn both shortened to BRO	Separate borough codes, BRX and BKN

4 Experiments

Each owner evaluated their component against acceptance targets agreed before building, on data the component never trained on, and compared the chosen method with at least one simpler baseline. The system as a whole was then measured by switching components off one at a time. Every number comes from a results file in the repository (folder `03_Research`) or from the component's own research report.

4.1 Results at a glance

Table 8. Headline results per component

Component	Test set	Result	Target or baseline
Intake bot	308 hand written messages	Macro F1 0.960, slot span F1 0.723, 78.5 ms per message	0.85 and 0.60, under 200 ms
Emergency check	Hand written emergency lines	Recall 1.000, 0 missed	Recall 1.00, no emergency filed
Credibility and escalation	1,121 synthetic reporters with hidden honesty	Spearman 0.914 over 1,121 citizens	0.85 for 10 or more reports
Predictive alerting	Daily demand with weather, held out	MAE 60.4 against 72.3, 87% of 23 storm spikes caught	Beat the no weather baseline, catch 70%
Duplicate detection	295 hand checked real pairs	Precision 0.90, recall 0.95, F1 0.92, false merges 0.10	Old method F1 0.69
Fake request detection	5,390 held out requests	AUC 0.61, false positives on first time reporters 0.33	Rules alone AUC 0.57
Admin questions	74 gold questions written before any strategy ran	Intent accuracy 0.96, unsafe requests refused 12 of 12, 3.6 ms	ML only 0.93

Agency and skill routing	49,558 historical records	Catalog agency matched 99.996%, referral advisory for 3 codes	Learned classifiers changed 0 assignments
External factors	Historical requests, 4,445 facilities	100 m threshold, 27.3% flagged, 90 of 90 tests, 0.14 ms	Threshold from a pre declared protocol
Urgency	672,041 policy labelled rows	Learned models imitate the policy (macro F1 up to 1.00), so rules ship	Majority baseline 0.28
Prioritisation	15 behaviour tests	All pass: ageing lifts old low requests, held duplicates last	Written by the owner
Crew deployment and routing	50 held out episodes, 5 rosters	9 to 18% more served and 38 to 48% less travel than rank first greedy, optimal on 60 of 60 small cases	Rank first greedy

4.2 Intake bot

Choice. A keyword baseline, TF IDF with an SVM, MiniLM sentence embeddings with logistic regression and a fine tuned transformer were trained on four variants of the training data, the transformer on a GPU. The embedding model won on accuracy for cost and ships: macro F1 0.960 on the team's test lines and 0.731 on 156 lines written to avoid obvious keywords. Below a confidence threshold it abstains and a person decides. A fine tuned token tagger reads the address and time with span F1 0.723 on 308 hand marked messages, against 0.434 for hand written patterns.

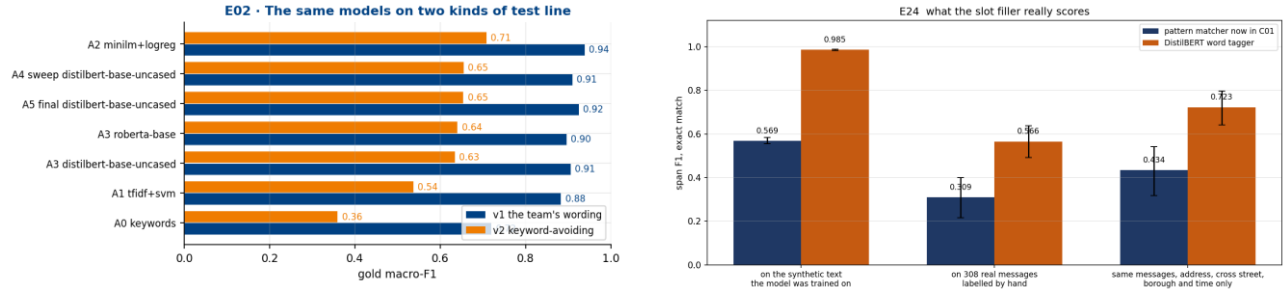


Figure 10. Intake bot: request type accuracy on two test slices (left), slot span F1 against the pattern baseline (right)

Left: the gap between the two slices shows how much an easy test set flatters a model. Right: the learned tagger beats the pattern baseline.

4.3 Emergency check

Choice. Weighted lexicon rules with negation handling, with a TF IDF classifier allowed only to add a detection, never remove one. Recall 1.00: no emergency in the test set was filed. Rather than lower the threshold until hydrant and fire escape complaints raise false alarms, the bot asks the resident one question, "Is anyone in danger right now?", when unsure. If the component fails, the 911 advice is shown anyway.

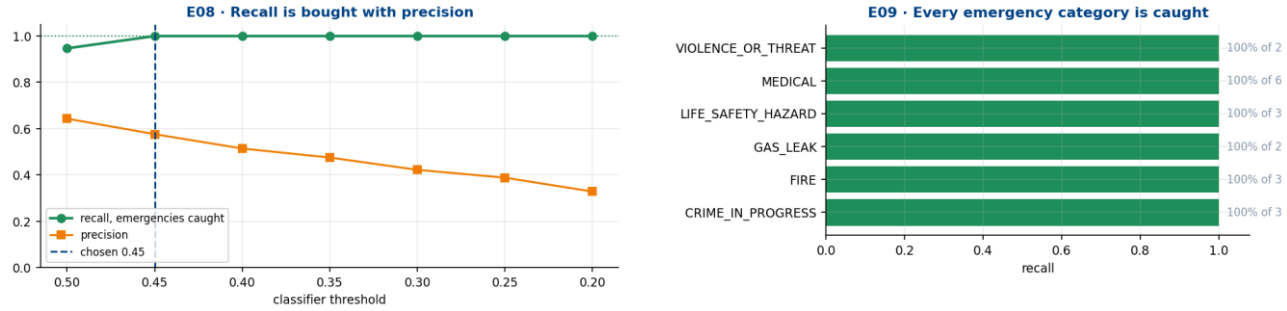


Figure 11. Emergency check: recall and false alarms across thresholds (left), recall by emergency category (right)

Left: no single threshold reaches full recall without false alarms, which is why the check asks a question instead. Right: every category is caught.

4.4 Credibility and escalation

Choice. A Beta distribution trust model with time decay, chosen over a simple point counter. Each reporter is compared with the base rate of the request types they report, so honest people who report hard to confirm problems are not punished. Spearman 0.914 against each synthetic reporter's hidden honesty, and the unfair gap between hard and easy report types fell from 4.0 to 0.3 points. A rule engine recommends escalation for problems that keep returning at the same place, and a person approves it.

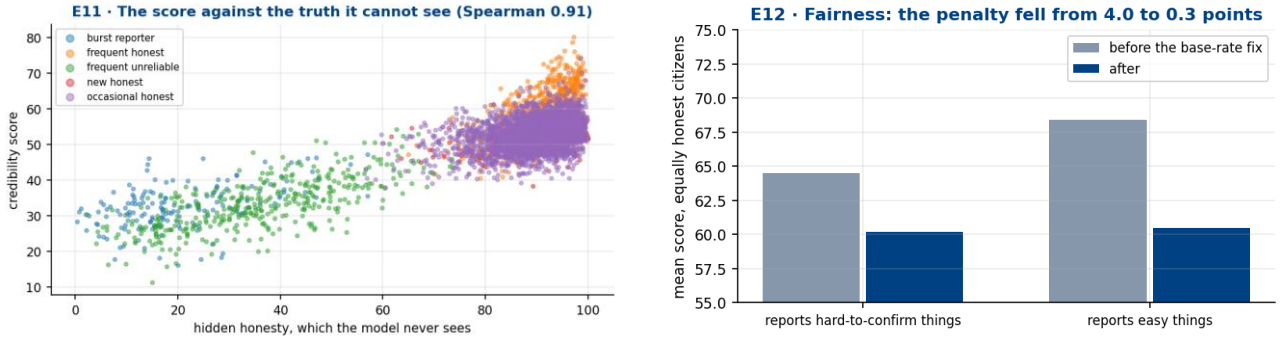


Figure 12. Credibility: score against each reporter's hidden honesty (left), and fairness across report types (right)

Left: the score rises with true honesty, so it ranks reporters correctly. Right: after the base rate correction, report type no longer moves the score.

4.5 Predictive alerting

Choice. Gradient boosting with a Poisson loss and weather features, compared with a no weather seasonal baseline and a Poisson regression. Count error 60.4 against 72.3 without weather, and 87% of 23 storm day spikes caught. A failure shows as failed on the dashboard, never as a quiet day.

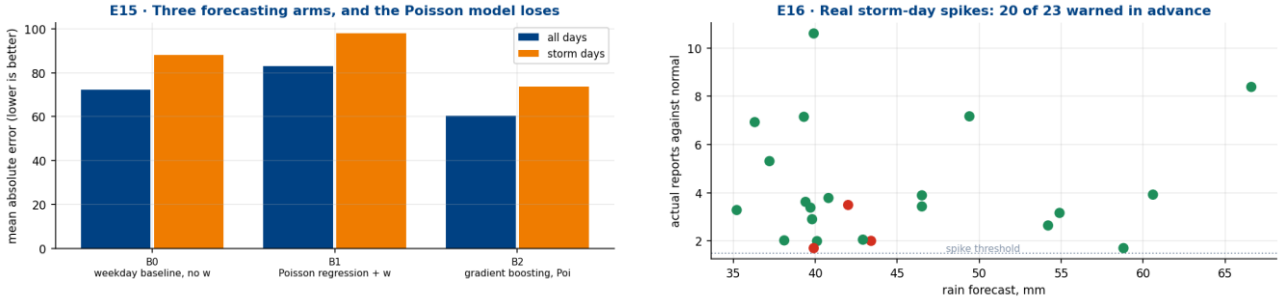


Figure 13. Predictive alerting: error of each forecasting approach (left), storm day spikes caught (right)

Left: weather features cut the error against the seasonal baseline. Right: most real spikes on storm days were flagged in advance.

4.6 Duplicate detection

Choice. Eight methods and one ablation were compared on 295 real pairs checked by hand: exact keys, radius and time windows, fuzzy addresses, DBSCAN, HDBSCAN, a graph method and two hybrids. The tuned evidence hybrid ships: precision 0.90, recall 0.95, F1 0.92 and an adjusted Rand index of 0.91, against F1 0.69 for the first version, in 0.02 ms per request. A problem reported again after its request was closed is marked a recurrence, not a duplicate.

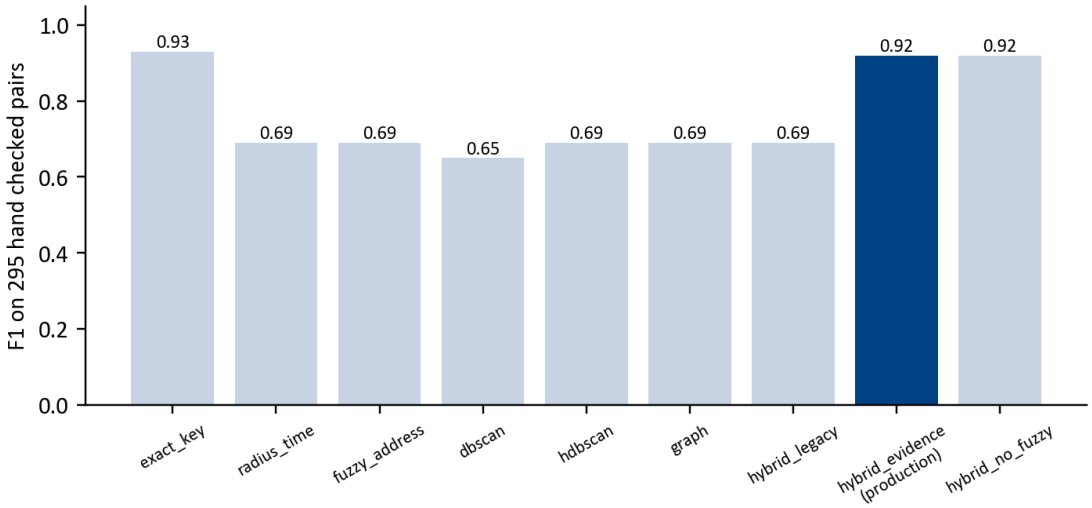


Figure 14. Duplicate detection: F1 of each method on the same 295 hand checked pairs

Clustering alone merges separate incidents. Requiring evidence from time, distance and address together nearly halves false merges.

4.7 Fake request detection

Choice. Rules, logistic regression and gradient boosting were compared with calibration on 5,390 held out requests. The hybrid of rules and calibrated logistic regression ships because it is explainable and keeps false positives on first time reporters at 0.33. Its separation is modest (AUC 0.61), so by design it only lowers a report or asks a person to look, and never rejects one.

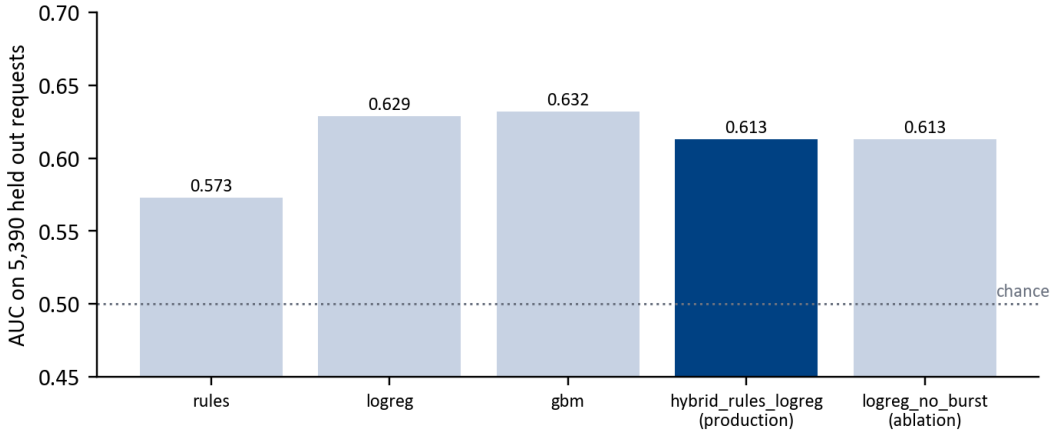


Figure 15. Fake request detection: AUC of each method on 5,390 held out requests

All methods sit just above chance, which is honest evidence that an unfounded report is hard to spot from the report alone.

4.8 Admin questions

Choice. Staff ask questions about the live queue in plain words. Rules, a TF IDF intent classifier and a hybrid were benchmarked on 74 gold questions written before any strategy ran. The hybrid ships: intent accuracy 0.959 with confidence interval 0.91 to 1.00, every answer grounded in the data, and all 12 attempts to change data refused. It never generates SQL, so it can only read.

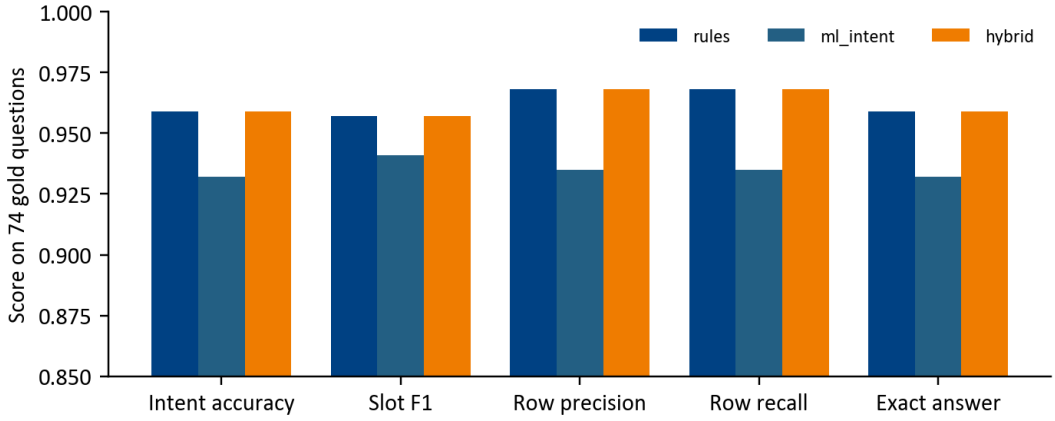


Figure 16. Admin questions: three strategies on 74 gold questions

Rules and the hybrid tie, both ahead of learning alone. The hybrid keeps the learned fallback for phrasings the rules miss.

4.9 Agency and skill routing

Choice. The question was whether routing needs learning at all. In 49,556 of 49,558 historical requests (99.996%) the agency matched the catalog, and learned classifiers per code and per borough changed no assignment. So the catalog rules ship. Learning did find one useful pattern: 503 of 1,164 referred requests fall in three codes, so the component adds an explainable advisory when a hand off between agencies is likely.

Table 9. Agency and skill routing: approaches compared

Approach	Changes against the catalog	Decision
Catalog lookup by request type	None by definition	Ships
Majority classifier per code (C1)	0 assignments	Evidence that rules suffice
Majority classifier per code and borough (C2)	0 assignments	Evidence that rules suffice
Referral advisory rule from training data	Advisory only, never reassigns	Ships beside the catalog

4.10 External factors

Choice. Straight line distance from the request to 4,445 critical facilities from OpenStreetMap (schools, hospitals and similar). The 100 m threshold was chosen on validation data under a protocol written before testing, and flags 27.3% of historical requests. 2.5% of requests lack a usable location and return null, never zero. 90 of 90 tests pass, at 0.14 ms per request.

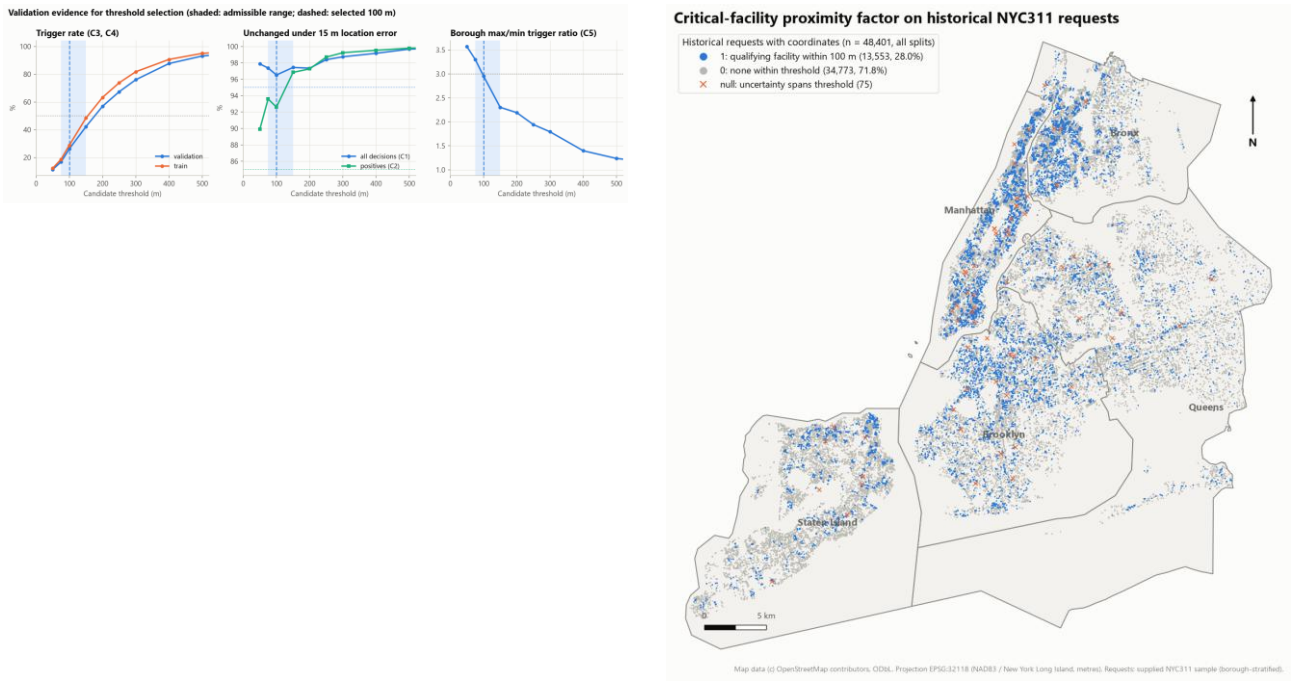


Figure 17. External factors: threshold candidates on validation data (left), requests flagged across the city (right)

Left: the candidate thresholds compared on validation data, from which 100 m was selected under the protocol. Right: where flagged requests fall across the five boroughs.

4.11 Urgency and Prioritisation

Choice. Urgency uses forward chaining rules over a hazard knowledge base and service impact rules. Four learned model families were also trained on 672,041 labelled rows and reached macro F1 up to 1.000, against 0.281 for the majority baseline. Such a score means the models had learned the labelling policy itself, not new knowledge, so the transparent rules ship. Prioritisation ranks every open request with a weighted score: waiting time, SLA risk, approved escalations, confirmed sensitive sites and extra reporters, with ageing so old low requests are not starved. Held duplicates go last. Its 15 behaviour tests pass. The SLA risk comes from Khalid's table of 3,563 official NYC 311 service deadlines. His code uses a deadline only once a reviewed mapping links a request type to its official category. He reviewed the water and sewer types, and we mapped 27 of the 34 types in all from the most common real descriptor in our 311 sample, so the SLA weight now works for most requests.

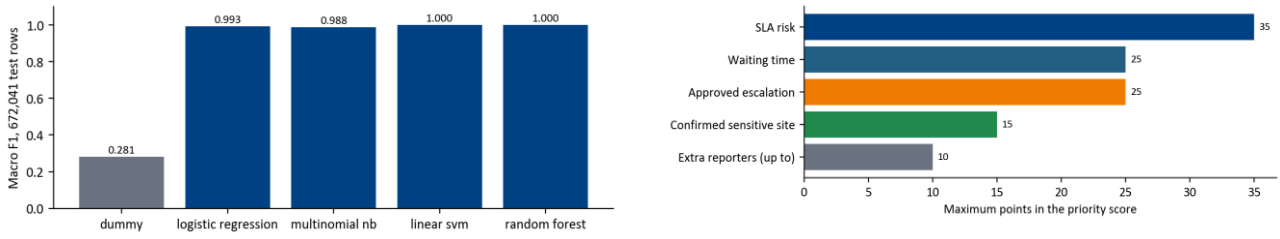


Figure 18. Urgency: macro F1 of each learned model (left), and the maximum points each factor adds in Prioritisation (right)

Left: near perfect scores reveal policy imitation, which is why rules ship. Right: SLA risk carries the most weight, then waiting time and escalation.

4.12 Crew deployment and route optimisation

Choice. A genetic algorithm assigns and orders jobs for each crew, with A* search using ALT landmarks for travel times on the NYC road network. It was compared with dispatching strictly in queue order (rank first greedy), a travel

aware greedy and an exact solver on 50 held out episodes across five crew rosters. It serves 9 to 18 percent more requests than rank first greedy with 38 to 48 percent less travel, and finds the exact optimum on 60 of 60 small cases. A travel aware greedy drives slightly less but breaks the queue's priority order far more often. A* returned exactly Dijkstra's cost on 500 random pairs while expanding 12.8 percent of the nodes, and rolling replanning cut travel 7.6 percent against a static plan while its stability rules halved plan changes.

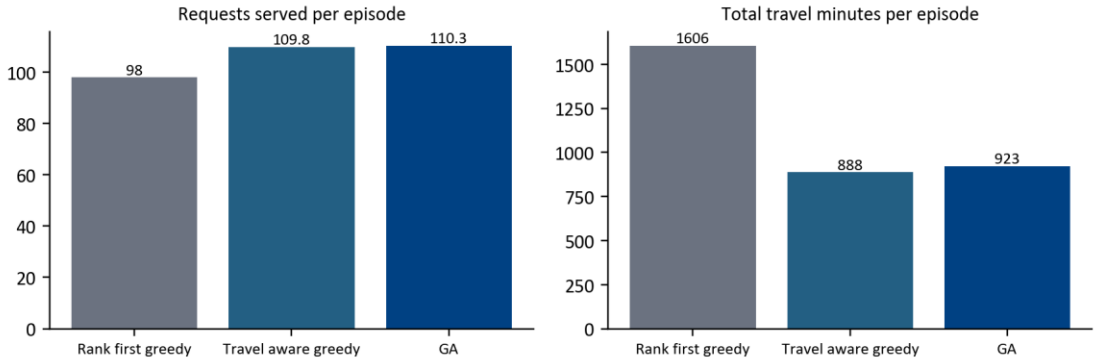


Figure 19. Crew deployment: the genetic algorithm against two greedy baselines, capacity factor 1.0

The genetic algorithm serves the most requests while keeping travel near the travel aware greedy's.



Figure 20. Crew deployment: methods across crew capacity (left), and one planned day of routes on real roads (right)

Left: the genetic algorithm's advantage is largest when crews are short. Right: each colour is one crew's ordered stops.

4.13 The experimentation journey

Several results changed our design. The table lists the turns that mattered, in the order they happened.

Table 10. Experiments that changed a decision

Experiment	What we found	What we changed
Intake: keyword, TF IDF, embedding and transformer models on four training variants	Sentence embeddings with logistic regression beat TF IDF and a fine tuned transformer on accuracy for cost	Shipped the small embedding model, under 80 ms per message on a CPU
Intake: our own 156 test lines that avoid keywords	Scores dropped from 0.96 to 0.73 on lines written to avoid obvious words	Reported both slices, and added a slot tagger trained on hand marked spans
Emergency: threshold sweep	Any threshold that kept recall at 1.00 also raised false alarms on hydrants and fire escapes	Ask the resident one question when unsure, instead of a lower threshold
Credibility: fairness by request type	Hard to confirm problem types cost honest reporters 4 points	Corrected for each type's base rate, now 0.3 points
Duplicates: 295 pairs checked by hand	The first matcher merged half of the separate incidents (F1 0.69)	Tuned an evidence based hybrid (F1 0.92)
Urgency: four learned model families	Near perfect scores, because the labels were the	Kept the explainable forward chaining rules

	catalog policy itself	
Crew planning: objective chosen on validation episodes only	A ranked objective protected priority better than a weighted sum	Froze it before testing on 50 held out episodes
System: switching each component off	At first three components left the queue unchanged	Found two integration faults and a missing staff step, then fixed them

4.14 Does each component matter? Switching them off

To see each component's effect on the whole system, we filed 34 hand written scenarios, with every fourth reported again by a second resident (43 cases), first with everything on and then with one component at a time on its safe default. A dispatcher's sign offs were applied, because Prioritisation counts escalations and sensitive sites only once a person approves them.

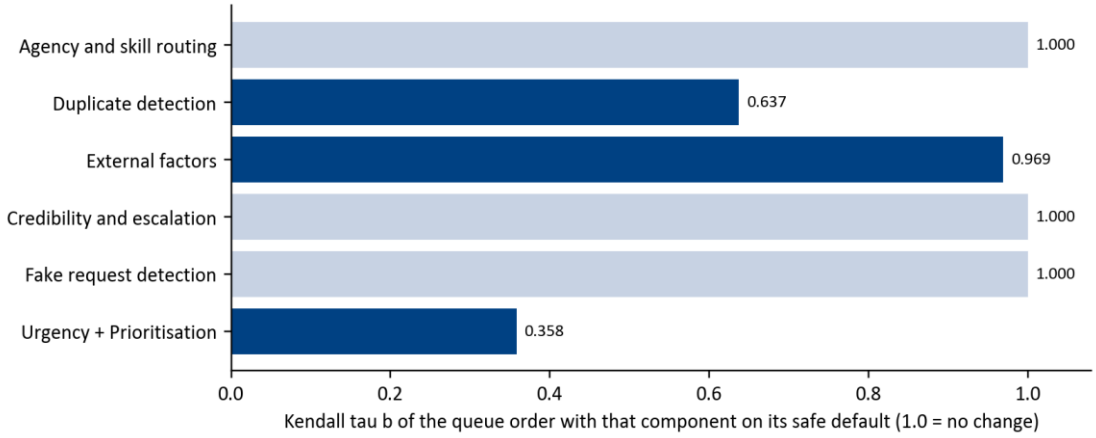


Figure 21. Queue order with each component on its safe default, as Kendall tau b against everything on

Bars below 1.0 mean switching that component off changes who is served first.

Duplicate detection and External factors visibly move the queue. Agency routing does not move it, but without it 40 of 43 cases lose their dispatch decision. Credibility changes the fake risk action on 23 cases but not the order, because it reaches the queue only through an escalation, and no test reporter has the repeated closed reports that trigger one. Fake request detection changes its own action on 29 cases and never the order, by design: it flags reports for a person and never rejects one.

4.15 Testing

- 168 automated tests across the app and the Python components, plus each owner's own suites (for example 83 for Crew deployment, 90 for External factors).
- 48 browser test flows run at phone, tablet and desktop size, checking every role, every screen and layout faults such as text cut off or content sticking out.
- A fresh clone from GitHub was set up from the written guide and run end to end before each release.

5 Findings and discussion

5.1 What worked

- The contract first approach. Agreeing inputs, outputs and safe defaults before building let four people work separately and still integrate twelve components in days.
- Rules and learning where each fits. Learning won where language is messy (intake) or patterns are statistical (duplicates, demand). Rules won where the data is policy (urgency, agency), and the experiments showed why.

- Measuring inside the system. Component scores alone hid three integration faults that the switch off measurement exposed.

5.2 Limitations

- Synthetic people and crews. Reporter histories and crew rosters are generated, so credibility and crew results show the method works, not its real world value.
- Credibility does not move the test queue, because no test reporter has the repeated closed reports that make it recommend an escalation.
- Fake request detection separates genuine from unfounded reports only modestly (AUC 0.61), so it flags for review and never rejects.
- Seven of the 34 request types have no official SLA in the published NYC table that maps cleanly, so for them the SLA weight is zero.
- The system runs on one machine. A public deployment needs a host, HTTPS and a managed database.

5.3 Ethics

A system that scores residents can treat them unfairly. Credibility starts every new reporter at exactly neutral, one bad report cannot drop a trusted reporter into the low band, and reporting hard to confirm problems costs under 0.3 points. Fake request detection is limited to asking a person to look. Staff never see who reported a request, and the resident never sees any score.

5.4 Future work

- Agency confirmed SLA categories for the remaining seven request types, and per request descriptors instead of one default per type.
- A pilot with real crew rosters and a full day of approvals, comparing planned routes with hand assignment.
- Proper hosting, with resident sign in by a one time code.

6 Conclusion

Civic Operations shows that city service triage can be both automated and explainable. Twelve components built by four people work together as one service, each with a measured effect and a safe failure, and people keep the decisions that carry risk. The most useful lessons came from integration and honest measurement rather than from any single model.

References

[1] NYC Open Data. 311 Service Requests from 2010 to Present. https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2010-to-Present/erm2-nwe9

[2] OpenStreetMap contributors. Map data, ODbL 1.0. https://www.openstreetmap.org/copyright

[3] Hart, P., Nilsson, N. and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics.

[4] Goldberg, A. and Harrelson, C. (2005). Computing the shortest path: A* search meets graph theory (ALT). SODA.

[5] Reimers, N. and Gurevych, I. (2019). Sentence BERT: sentence embeddings using Siamese BERT networks. EMNLP.

[6] Josang, A. and Ismail, R. (2002). The Beta reputation system. Bled eConference.

[7] Ester, M. et al. (1996). A density based algorithm for discovering clusters (DBSCAN). KDD.

[8] SeeClickFix (CivicPlus) and CitySourced (Granicus) product documentation, reviewed September 2026.

[9] Civic Operations source code and research files. https://github.com/DevelopingGod/IRS-Project-2026-KIMS

Glossary

Term	Meaning
311	New York City's number and service for non emergency requests to city agencies.
A* search, ALT	A shortest path search guided by an estimate of the remaining distance. ALT uses landmarks to sharpen that estimate.
Adapter	The thin piece of code that connects a teammate's component to the app without changing its logic.
AUC	Area under the ROC curve: how well a score separates two classes. 0.5 is chance, 1.0 is perfect.
Beta trust model	A credibility score built from counts of confirmed and unfounded reports, starting neutral.
Brier score	The mean squared error of predicted probabilities. Lower is better calibrated.
Docker container	A packaged program with everything it needs, so it runs the same on any computer.
Escalation	A recommendation to treat a recurring problem more seriously. It counts only once a person approves it.
F1, precision, recall	Recall is the share of true cases found, precision the share of flags that are right, F1 balances both.
Genetic algorithm	A search that improves a population of candidate plans by selection, crossover and mutation.
Intent and slot	The kind of request a message describes, and the details it gives, such as the address or time.
Kendall tau b	Agreement between two orderings. 1.0 means the same order.
MR, RS, CGS	Machine Reasoning, Reasoning Systems and Cognitive Systems, the three courses this practice module draws on.
OSM	OpenStreetMap, the open map data used for facilities and roads.
Proposal only	The crew planner suggests routes but never assigns. A person approves.
Safe default	The agreed output a component gives when its code is missing, fails or is switched off.
SLA	Service level agreement: the target time an agency sets for a request type.
Spearman correlation	Agreement between two rankings of the same items.
Synthetic data	Generated data, labelled as such, used where real data cannot exist, such as reporter histories.

Appendix A Functions mapped to MR, RS and CGS

Table 11. System functions against the course knowledge and techniques

Function	Machine Reasoning (MR)	Reasoning Systems (RS)	Cognitive Systems (CGS)
Intake bot		Supervised classification, abstaining when unsure	Intent detection, slot tagging, sentence embeddings, dialogue
Emergency check	Weighted rules, fail closed	Threshold chosen on a recall first sweep	Lexicon based language screening
Agency and skill routing	Catalog knowledge base, business rules	Learned baselines as evidence for the rules	
Duplicate detection	Matching rules over evidence	Clustering (DBSCAN, HDBSCAN), grid search tuning	Fuzzy address matching
External factors	Spatial rules, knowledge representation of facilities	Threshold sensitivity analysis	
Credibility and escalation	Escalation rule engine	Beta distribution model, reasoning under uncertainty	
Fake request detection	Guardrail rules	Calibrated logistic regression, gradient boosting compared	
Urgency	Forward chaining over hazard knowledge bases	Four learned model families compared	
Prioritisation	Weighted multi criteria decision rules	Ranking evaluation	
Crew deployment and routing	Genetic algorithm, A* informed search, constraints	Benchmarks against greedy baselines	
Predictive alerting		Demand forecasting with weather features	
Admin questions	Read only query rules, refusal of write requests	TF IDF intent classifier	Natural language questions over data

Appendix B Installation and user guide

Installation

Needs Git and Docker Desktop. Every component, the data and the models are in the repository.

```
git clone https://github.com/DevelopingGod/IRS-Project-2026-KIMS.git
cd IRS-Project-2026-KIMS/01_Application
docker compose up --build -d
docker compose exec backend python -m app.backend.manage import-backlog --count 150
docker compose exec backend python -m app.backend.manage team-accounts > credentials.txt
```

Open http://localhost:8080. The first build takes 10 to 20 minutes. `credentials.txt` holds the test IDs and PINs for city staff, crews and the tech team. `SETUP.md` in the repository covers troubleshooting and running without Docker.

Using it

Role	How to sign in	What to try
Resident	Home page, answer Yes at the end, register with any email	Report a pothole by form or in plain words, then track it and ask the status bot
City staff	Footer link, staff ID and PIN	Filter the queue, open a request and read why, confirm a school impact, approve a crew route
Crew	Footer link, crew ID and PIN	See approved stops on the road map, close a job with what you found
Tech team	Footer link, tech ID and PIN	Switch a component off and see staff told, run the measurement, create accounts

Appendix C Project proposal

The proposal as submitted on 12 September 2026 follows on the next pages. Changes since: the Profanity filter and Resolution time were dropped as optional, a fourth portal was added for the tech team, and the crew planner works as a proposal that staff approve.

PROJECT PROPOSAL

Name of Topic:

Civic Operations: Explainable City Service Triage and Field Crew Dispatch

NUS MTech in AIS, Intelligent Reasoning Systems

Field	Detail
Date of Proposal	12th September 2026
Project Title	Civic Operations: Explainable City Service Triage and Field Crew Dispatch
Group Number	23
Group Members	Khalid Alhazmi **[A0361128J]** Mohammad Alharthi **[A0354097W]** Nabeel Taresh **[A0352296X]** Sankalp Sandeep Indish **[A0351383E]**
Sponsor / Client	Not applicable. This is an internally driven academic project, no external sponsor is confirmed.

1. Introduction

We decide to build Civic Operations Project to help a city operations team decide which non emergency service requests to handle first, with which crew, and why. We deliver it as three connected applications over one shared backend, a citizen portal for filing requests by form, voice, or a conversational bot, a crew portal for field workers to report the true outcome of each job, and a municipality portal giving administrators a live operational dashboard. The system predicts urgency and effort, understands if any external factors (such as close proximity to locations that are critical), applies the city's own policy rules, groups repeated and cross channel reports of the same incident, screens for requests that are not really there, forecasts likely demand from live weather conditions, and assigns and routes crews efficiently, explaining every decision in plain language. Our goal is a system a real city could trust, faster resolution of genuinely urgent problems, a shrinking backlog, fair treatment of citizens, and decisions that an auditor can understand.

2. Project Background and Market Context

Non emergency complaints are handled through 311 systems, kept deliberately separate from 911 emergency lines since **Baltimore's 1996 rollout**, so routine issues such as potholes, blocked drains, and noise do not compete with life safety calls.

Existing 311 software, including SeeClickFix (CivicPlus) and CitySourced (Granicus), focuses on multi channel intake and category based routing, with escalation, where it exists, implemented as a service level timer rather than a genuine check for a recurring problem.

In researching this space, we found that none of the platforms or papers that we reviewed chain a prediction into a policy layer, into duplicate and fake screening, into crew and route optimization, into a citizen facing status loop, the way we do here. That combination is our market position.

3. Literature Review and Market Research

What Was Researched	What Was Found	What Does Not Exist Yet
SeeClickFix, CitySourced, and Accela class 311 CRMs, IBM 311 capstone projects, Scaling the Queue (arXiv 2605.06482, 2026), and an open data curation study (arXiv 2502.08649, 2025).	Commercial platforms stop at intake, routing, and time-based escalation. Every research project reviewed stops at one modelling stage, classification, resolution time prediction, or reinforcement learning based routing.	No system reviewed chains prediction into policy rules, duplicate and fake screening, crew and route optimization, and a citizen facing explanation loop together. No system reviewed keeps a recoverable, per citizen credibility score inside a municipal complaint pipeline, the closest real analog found is Facebook's hidden content moderation trust score, from an unrelated domain.

4. Project Scope

We commit to all 4 technique groups as real, working deliverables. Decision Automation runs the rule engines behind escalation, eligibility, and duplicate handling. Resource Optimization assigns crews and plans routes with informed search and a genetic algorithm. Knowledge Discovery predicts urgency, effort, and resolution time, understands if there are any external factors adding to decision and detects recurring hotspots and fake reports. Cognitive Systems is delivered as the NLP intake bot and the admin facing natural language question answering layer.

In Scope

- **Three connected applications:** a citizen portal, a crew portal, and a municipality admin portal, sharing one backend and one database.
- **Five working areas**, **not one borough alone**, so the system is demonstrably not static or single area based, using up to 10,000 real NYC 311 rows per area, topped up with synthetic data where more volume or history is needed.
- **An explicit emergency safeguard:** any request whose content suggests a medical, police, or criminal matter is stopped before filing, with a prominent on-screen instruction to contact 911 instead.

Out of Scope for This Phase

- Any live integration with an actual city agency or a real crew workforce. All crew and citizen data used for demonstration is synthetic and clearly labelled as such.
- A final, fixed choice of paid infrastructure. Hosting for the Python backend service and the database is described in Section 6 as still being evaluated among free options, and will be finalized during the build.

Note: *If we cannot complete any part of this scope by the final deadline despite this plan, we will report that honestly in the final submission's Challenges and Roadblocks section, rather than hide it or quietly drop it.*

5. Data Collection and Preparation

Our primary source is the NYC 311 Service Requests dataset, 2020 onward, drawn from 5 working areas rather than one borough, with up to 10,000 real rows pulled per area directly from NYC Open Data at no cost. We deliberately changed this from an earlier, larger single area pull, so the working dataset stays small enough to version, fast to iterate on, and genuinely multi area rather than static. **Where we need additional volume, history, or coverage beyond that real sample, we use synthetic rows and label them clearly as synthetic** throughout the system and the report, never presented as if they were real 311 records.

Synthetic Data, Generated Once and Shared

Several components need synthetic data the public dataset cannot provide, citizen and reporter identity, crew rosters, resolution time labels, and urgency scenario labels. **Every teammate's own worked upon and documented knowledge independently converges on the same requirement**, that we generate these synthetic populations once and share them across components rather than each of us regenerating our own, so that a synthetic citizen's credibility score and their duplicate or fake reporting history refer to the same person throughout the system.

6. System Design

We organize **the system as three portals over one shared reasoning pipeline**.

A citizen files a request by web form or by speaking to a lightweight NLP bot, both are screened for profanity and for emergency content before anything is stored. Every request then passes through duplicate and fake screening, urgency assignment, escalation and credibility scoring, external factors applicability checks and prioritization in near real time, while crew assignment and route optimization run periodically as a batch planning step over all open, prioritized requests.

A crew portal lets field workers report the true outcome of each job, closing the loop back to the citizen's own status view and to credibility scoring.

An admin portal gives the municipality a live operational dashboard. The full diagram is on the following page.

Component Ownership and Technique Mapping

Component	Owner	Core Technique	Technique Group
Duplicate & Cross-Channel Detection	Nabeel Taresh	Exact key match, DBSCAN/HDBSCAN, NetworkX grouping	Knowledge Discovery, Decision Automation, Cognitive Systems
Fake & Unfounded Request Detection	Nabeel Taresh	Rule guardrail, logistic regression, gradient boosted trees	Decision Automation, Knowledge Discovery
Escalation & Citizen Credibility Scoring	Sankalp Indish	Rule engine, Beta distribution trust model	Decision Automation, Knowledge Discovery
Urgency Assignment	Khalid Alhazmi	Forward chaining rules, weighted scoring	Decision Automation

Request Prioritization	Khalid Alhazmi	Weighted multi criteria score, genetic algorithm	Decision Automation, Resource Optimization
Resolution Time & External Factors	Mohammad Alharthi	Synthetic resolution time regression, OSM proximity	Knowledge Discovery, Cognitive Systems
Crew Deployment & Route Optimization	Mohammad Alharthi	A* informed search, genetic algorithm	Resource Optimization
NLP Intake Bot	Sankalp Indish	Lightweight intent and slot filling	Cognitive Systems
Advanced Predictive Alerting	Sankalp Indish	Weather cross referenced pattern matching	Knowledge Discovery
Profanity Filter	Khalid Alhazmi	Lexicon and classifier-based text screening	Decision Automation, Knowledge Discovery

Note: *Full field level inputs, outputs, business metrics, benchmarking plans, and evaluation metrics for every component above are documented in the shared component template maintained in the project repository, so the detail behind each row is not lost, only kept out of this table.*

How This Works, One Example

Picture a single request during a storm.

A citizen who cannot type tells our NLP intake bot by voice that a drain outside their home is flooding. The bot fills in the request, and the profanity filter finds nothing to flag.

The emergency check confirms this is not a medical, police, or criminal matter, so it is not redirected to 911.

Duplicate and cross-channel detection checks whether anyone nearby has already reported the same flooding, in this example nobody has, so the request stands on its own.

Escalation and credibility scoring checks the citizen's trust score, a returning citizen with a good track record starts fully trusted, so no extra scrutiny is applied, and fake and unfounded request detection finds nothing suspicious.

The external factors and resolution time component work towards understanding parameters if the request has been raised to a critical location such as an area with closer proximity to a hospital. This helps add to knowledge of resolution time and prioritization in linear, directed way.

Meanwhile, advanced predictive alerting has already told the admin dashboard, from live weather data, that heavy rain in this area makes drainage complaints likely today, so this request confirms rather than surprises that forecast.

Urgency assignment reads the storm context and the complaint type and assigns a high urgency level, and request prioritization places it near the top of today's queue.

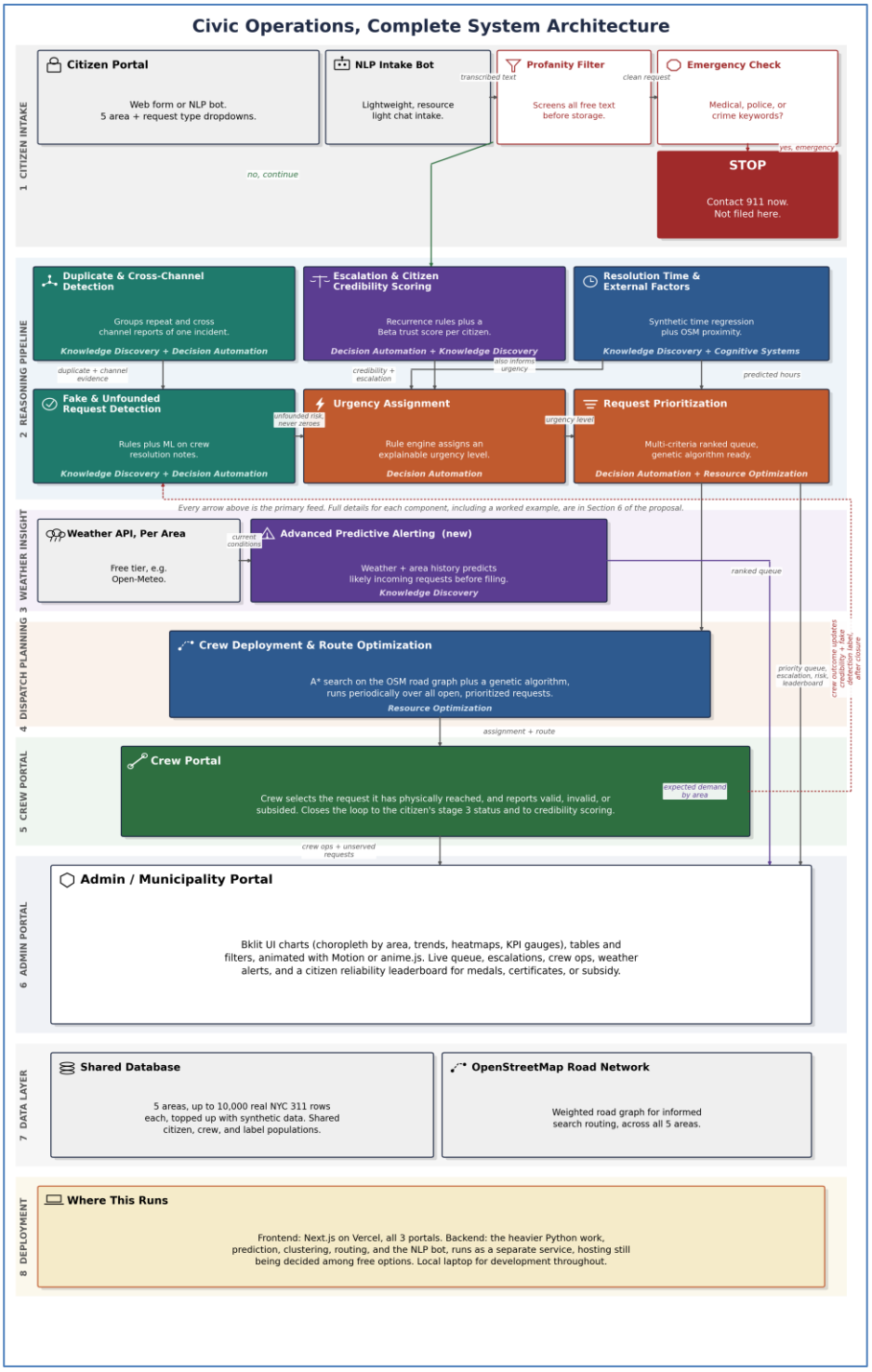
In the next batch planning run, crew deployment and route optimization assigns the nearest qualified crew and plans their route.

The citizen sees their request move to stage two, processed, with an estimated resolution time and the crew type assigned.

Once a crew clears the drain, they log the outcome as valid in the crew portal, the citizen's status moves to stage three, resolved, and credibility scoring records another confirmed genuine report, keeping this citizen's trust score high for next time.

System Architecture Diagram

Note: *A full resolution version of this diagram is also available in the project repository.*



7. Evaluation Metrics

We evaluate this system on two levels, whether each component performs well on its own technical terms, and whether the system as a whole actually helps the city. A metric is only useful if it is matched to the kind of output it is judging, so the tables below name a specific metric for each situation rather than a generic accuracy figure.

Technical and AI Metrics

Component or Task	Metric	What It Measures
Duplicate and cross-channel detection	Precision, recall, F1 score	How well true duplicate pairs are found without merging genuinely separate incidents
Duplicate and cross-channel detection	Silhouette score, Adjusted Rand Index	How well separated the spatio-temporal clusters are, and their agreement with a hand labelled sample
Fake and unfounded request detection	AUC ROC, Brier score	Separation between genuine and unfounded reports, and how well calibrated the risk probability is
Fake and unfounded request detection	Precision at K, false positive rate on first time reporters	Quality of the highest risk flags, and fairness to citizens with no history yet
Urgency assignment	Critical/High recall, false negative rate on Critical requests	The main safety metrics, catching genuinely urgent cases and never missing one
Urgency assignment	Weighted Cohen's kappa	Agreement with expert labels, penalizing a severe misclassification more than a minor one
Request prioritization	Precision at K, NDCG at K, Kendall rank correlation	Whether the most important requests actually surface near the top of the queue
Escalation and citizen credibility scoring	Mean absolute error against a synthetic citizen's hidden true honesty rate	How accurate the trust score is, checked against ground truth only we can see in a synthetic test
Escalation and citizen credibility scoring	False positive rate on new citizens	Fairness, a brand new citizen should never be judged on one report
Resolution time and external factors	Mean absolute error, RMSE, R squared	Accuracy of the predicted resolution time against held out data
Crew deployment and route optimization	Total distance, optimality gap, runtime	Efficiency and quality of the route plan against a simple nearest-crew baseline
NLP intake bot	Intent and slot filling accuracy	Whether the bot correctly captures what the citizen actually said
Advanced predictive alerting	Precision and recall of predicted demand against requests actually filed	Whether the weather based forecast genuinely matches what citizens report
Profanity filter	Precision, recall, false positive rate	Whether abusive language is caught without wrongly flagging clean text

Business Metrics

Metric	What It Shows
Percentage reduction in backlog	Whether recurring problems get solved for good instead of cycling through repeated temporary fixes
Average days to a permanent fix after escalation	Whether escalation actually speeds up a lasting resolution
Percentage reduction in total crew travel distance	The efficiency gained from optimized routing compared with a first in, first out baseline
Percentage of Critical and High requests acknowledged within target time	How responsive we are to genuinely urgent cases
Wasted crew trips avoided per month	The direct saving from catching fake or unfounded reports before a crew is sent
Percentage of genuine requests wrongly de-prioritized	The harm guardrail, tracked alongside the saving above so one is never improved at the other's expense
Citizens informed per resolved job	The value of grouping duplicates, one fix can now notify everyone who reported it
Context enrichment coverage	The percentage of requests where nearby hospital or school context was successfully attached
Number of citizens recognized on the leaderboard	How much the new recognition feature, medals, certificates, or subsidy, is actually being used
Percentage of predicted demand that matches requests actually filed	The real world value of the weather based alerting, not just a technical score

8. Implementation, Results, Challenges, Future Work, and Conclusion

These sections are optional at the proposal stage under the module guidelines and become mandatory in the final report, due 25 October 2026. We will document implementation progress, real experiment results against the metrics above, honest challenges encountered, and future work as the build proceeds, using the architecture and component ownership fixed in this proposal as the baseline plan against which we report our own progress and any deviations.

9. Supplementary Materials

Note on the Use of AI

This proposal, the system architecture diagram, and supporting project documents were drafted with AI assistance for formatting purposes.

Every technical decision, component ownership assignment, and scope commitment reflected here originates from the team's own component documentation and team discussions, the AI assistance was used for drafting, formatting, and synthesizing that material into this document, and for producing the architecture diagram from the team's specifications.

The team has reviewed and takes ownership of the content.

Acknowledgements

The team thanks the course instructors for the practice module guidance that shaped this proposal's structure.